Lucas De Boi (392831)
Daniel Lelescu (397627)

Semestre de printemps 2026

Projet MCU BA4

- Rapport Technique -

1. Introduction

Ce projet porte sur le développement d'une application réalisée en assembleur et implémentée matériellement sur la carte ATmega128. Notre application simule un ventilateur commandé par télécommande. Étant donné les limitations des périphériques disponibles sur le kit, la fonction de ventilation est représentée par la rotation continue d'un petit servomoteur muni d'un bras, jouant le rôle d'une hélice.

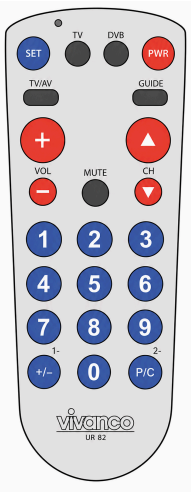
2. Description générale de l'application

Notre ventilateur présente deux modes distincts: manuel et automatique. Dans le mode manuel, l'utilisateur peut contrôler la vitesse du ventilateur par l'appui des boutons "1" à "9" de la télécommande infrarouge. Dans ce mode, le LCD affiche la vitesse choisie. En mode automatique, la vitesse du ventilateur est adaptée automatiquement en fonction de la température de l'environnement. Naturellement, plus il fait chaud, plus le ventilateur tourne vite. L'utilisateur peut modifier la température dite "cible" à partir de laquelle le ventilateur s'adapte dynamiquement. Ceci mimique l'utilisation d'un thermostate: l'utilisateur définit la température de la pièce, et si la température courante est au-delà de cette cible, le ventilateur ou la climatisation s'active afin de ramener la température de la pièce à la cible.

Le LCD affiche alors la température de l'environnement et la température cible.

3. Mode d'emploi

3.1 Contrôles par télécommande



<u>Changement de mode</u>

- **Channel UP**: mode manuel
- **Channel DOWN**: mode automatique

<u>Contrôles en mode manuel</u>

- **Chiffres 0-9**: vitesses du ventilateur
- **MUTE**: toggle pause/play

<u>Contrôles en mode automatique</u>

- **VOL+ :** shift vers le haut de la température cible/de référence
- **VOL- :** shift vers le bas de la température cible/de référence
- **MUTE:** toggle pause/play

4. Description technique de l'application

L'architecture de cette application se compose de deux modules d'acquisition de données, de trois modules d'exécution exploitant les données reçues, ainsi que deux modules intermédiaires assurant la communication entre ces différentes parties et la synchronisation des diverses opérations.

4.1 Modules d'acquisition de données (Input)

4.1.1 Module Infrarouge (IR) (ir_module.asm)

La télécommande IR constitue l'interface d'entrée utilisateur du programme. Il est donc essentiel que l'application puisse détecter en temps réel toute commande transmise par celle-ci. À cet effet, le détecteur infrarouge est relié à la broche associée à l'interruption externe INT7. Cette interruption est configurée pour réagir à un front descendant, afin de déclencher la routine d'interruption dès le début d'une transmission RC5 provenant de la télécommande et de se synchroniser avec celle-ci. La routine d'interruption échantillonne les bits du signal transmis et les stocke dans un "buffer" en espace mémoire données. Ceci fait, la routine se termine par lever un "flag" indiquant qu'une nouvelle commande brute a été réceptionnée. La commande utile en est alors extraite et exécutée plus tard dans le programme principal.

Branchement matériel du récepteur IR (module M2): **PORTE** , **Pin 7**

4.1.2 Module température (temp_module.asm)

Le module température pilote le capteur DS18B20, qui utilise le protocole de transmission 1-Wire. Il gère l'extraction périodique de la température environnante et son stockage en mémoire données sous forme de mot de 16 bits. Une particularité de ce capteur est que l'extraction de la température nécessite un temps d'attente d'autant plus long que la résolution de mesure est élevée. Pour éviter que le programme principal ne reste coincé dans une boucle d'attente pendant la durée d'une extraction, notre module met en œuvre une simple machine d'états (via une variable nommée **temp_state**) indiquant si le capteur est au repos (IDLE) ou effectue une conversion (CONV_RUNNING). En effet, en voyant que le capteur est en train d'exécuter une conversion, le programme principal peut traiter d'autres tâches entre temps. La période de lancement de conversion est géré par un simple flag temporel **temp_sched_flag** qui est mis à jour par le module scheduler (c.f. section 4.3.2). La durée restante d'une conversion en cours est stockée par une variable jouant le rôle d'un compteur logiciel, nommé **temp_conv_counter**. L'exécution d'une boucle du service de température suit la logique présentée dans le flowchart en **Annexe A**.

Il y a une particularité intéressante à mentionner. Quand une conversion est en cours, on vérifie si elle s'est terminée en regardant la valeur du compteur logiciel. Si celle-ci est non-nulle, elle doit être décrémentée. Or, la tâche de décrémenter le compteur logiciel n'est pas effectuée par le module température. Ceci est un choix conscient ayant pour but de respecter l'architecture que nous avons choisi d'implémenter, qui assigne toute gestion temporelle au module scheduler (c.f. section 4.2.2 pour plus de détails). De plus, lorsqu'une nouvelle température est disponible, on lève des flags de mise-à-jour propres aux périphériques dont le fonctionnement dépend de la température environnante (i.e. le servomoteur et le LCD, c.f sections 4.3.3 et 4.3.2 respectivement pour plus de détails).

Finalement, cette application implémente une température cible/de référence, stockée dans la variable **target_temp**. D'un point de vue architectural, cette variable apparaît plutôt comme une variable propre au module de gestion d'état, car celle-ci est uniquement manipulée dans la routine de mise à jour d'état appelée dans main, et non pas dans la routine du module température.

Au niveau du matériel, par construction du protocol 1-wire, le capteur de température doit être interfacé au microcontrôleur via un port bidirectionnel. Ceci élimine déjà les ports C et F. Le port B est réservé au servomoteur (c.f section 5.3) et le port A n'est pas disponible. Ceci laisse donc uniquement le port D

Branchement matériel du capteur de température (module M5): **PORTD, Pin 5**

4.2 Modules intermédiaires (Processing)

4.2.1 Module de gestion d'état (state_manager.asm)

Comme l'indique le nom du fichier correspondant, ce module gère l'état du système. Son rôle principal est de lire toute nouvelle commande utile stockée dans la variable nommée **command**, et de modifier les variables du système en conséquence. L'arrivée d'une nouvelle commande prête à exécuter est communiqué via la variable binaire **new_exec_cmd_flag**, qui signifie en anglais "new executable command flag". Celle-ci est mise à jour dans le service de post-traitement du module IR lorsqu' une nouvelle commande utile (6 bits) aura été extraite de la commande brute de 14 bits stockée dans le buffer dédié. La logique d'exécution du service de mise-à-jour du système est décrite dans le flowchart présentée en **Annexe B.**

Bien que l'identification de la commande dans le module de gestion d'état soit réalisée ici par une chaîne de comparaisons successives, d'autres approches plus générales existent, comme l'utilisation d'une table de correspondance ou d'une table de saut associant à chaque commande une action ou une sous-routine dédiée. Ces solutions deviennent particulièrement intéressantes lorsque le nombre de commandes possibles est important, car elles évitent l'allongement d'une structure conditionnelle et s'exécutent en un temps constant. Dans le cadre de notre application, le nombre de commandes reste toutefois très limité. Le recours à une chaîne de comparaisons a donc été conservé, car il constitue ici une solution simple, lisible et facile à mettre au point, sans introduire de complexité supplémentaire inutile.

4.2.2 Module de temporisation (scheduler.asm)

Le module scheduler assure toute la gestion temporelle de l'application. Cette centralisation permet de séparer la logique fonctionnelle des contraintes de timing, ce qui rend l'architecture plus claire et plus modulaire. Il s'appuie sur le compteur matériel **Timer0**, configuré en mode **CTC** (*Clear Timer on Compare Match*). Avec les valeurs de prescaler et de registre de comparaison choisies dans scheduler_module.asm, le timer atteint périodiquement la valeur de comparaison puis repart de zéro toutes les **10 ms**. À chaque événement de ce type, une **interruption d'output compare match** est déclenchée. Cette période de 10 ms constitue l'unité de temps de base du système.

L'écran LCD et le capteur de température nécessitent toutefois des périodicités différentes. Le LCD est rafraîchi toutes les **100 ms**, tandis qu'une nouvelle commande de conversion est envoyée au capteur de température toutes les **1000 ms**. Chacun de ces deux sous-systèmes possède donc un compteur logiciel propre, stocké en mémoire données. C'est dans la routine d'interruption associée à l'output compare match de Timer0 que ces compteurs sont mis à jour. Lorsqu'un compteur atteint sa période, il est rechargé et le drapeau temporel correspondant est levé (**lcd_sched_flag** et **temp_sched_flag**), indiquant au programme principal qu'il doit soit rafraîchir l'affichage, soit lancer une nouvelle conversion de température. Ces flags sont alors lues par les routines de service des périphériques correspondants.

La routine d'interruption associée à l'output compare match de Timer0 agit également sur le compteur logiciel stockant le temps d'attente d'une conversion de température: s'il est non-nul, il le décrémente et sinon, il ne fait rien.
Veuillez vous référer au flowchart décrivant la logique d'exécution en **Annexe C.**

4.3 Modules d'execution (Output)

4.3.1 Buzzer module (buzzer_module.asm)

Ce module a pour rôle simple de confirmer la réception d'une nouvelle commande de la télécommande sur la carte avec un court bruit, un "beep" en anglais. Cette action est faite à travers la routine **buzzer_beep**, qui est appelée en **stage_manger.asm** après la lecture de **new_exec_cmd_flag**. En implémentant ce module, nous avons remarqué des problèmes avec l'implémentation de la variable **new_exec_cmd_flag**, notamment qu'elle était mise à 1 en répétition si un bouton sur la télécommande était maintenu. Son implémentation permet maintenant l'émission d'un beep qu'une seule fois lors d'un appui sur un bouton de télécommande.

Branchement matériel du buzzer (module M2): **PORTE, Pin 2**

4.3.2 Module d'affichage (lcd_module.asm)

Le module est séparé en 2 grandes parties: **Service et Drivers**. La routine **lcd_service** sert à parcourir la logique d'un mis à jour de l'écran LCD, présentée sous forme de flowchart en **Annexe D**. Comme les autres routines de catégorie **(...)_service**, elle est appelée dans la boucle de **main**.

Les drivers, **lcd_display_auto** et **lcd_display_manual**, appelés par la routine **lcd_service** suivant sa logique conditionnelle, se basent sur les fichiers **lcd.asm** et **printf.asm** (fournis par le professeur) pour effectuer le protocol UART afin d'afficher les informations relevantes à l'écran LCD. Le formatting sur l'écran LCD suit ce modèle, avec chaque table étant le résultat de **lcd_display_manual** et **lcd_display_auto** respectivement. En mode AUTO, "R" signifie "Reference" en anglais.

"mode: MANUAL"
"speed: " **manual_speed**

"mode: AUTO R=" **target_temp_dec**
"temp: " **temp_raw**

C'est à noter que nous employons deux variables pour stocker la température cible: **target_temp** et **target_temp_dec**. **Target_temp** (mentionné en 4.1.2) est en format hexadécimal, pour être cohérent avec le output **temp_raw** de la module **temp_module.asm**. **Target_temp_dec** est en format décimal, destiné à l'affichage. Dans le format 11-bit de **temp_raw** (et donc de **target_temp**), une incrémentation de 0x10 correspond à une incrémentation de 1ºC en décimal.

En plus, vu que la **mode** au **reset** est initialisée à MANUAL, nous avons choisi d'appeler le driver **lcd_display_manual** avec les autres routines de catégorie **(...)_init**.

4.3.3 Module moteur (servo_module.asm)

Comme **lcd_module.asm**, ce module est séparé en plusieurs parties à cause de sa complexité: **Init, Service, et Drivers**. Ici la routine **servo_init** est nécessaire pour initialiser le compteur matériel **Timer1**. Ce compteur est configuré en mode **Fast PWM avec ICR1 comme valeur TOP**, avec le but de générer un signal carré de période fixe sans une routine de service d'interruption sur **PORTB, Pin 5**, avec un Pulse-Width en fonction des contenus du registre OCR1A.

La routine **servo_service** suit la logique d'un mis à jour du moteur servo, présentée sous forme de flowchart en **Annexe E**. Cette routine est appelée dans la boucle de **main**, comme les autres routines de service. En mode MANUAL, **servo_service** utilise un lookup table (avec le label **servo_speed_table**) pour associer un **manual_speed** à une valeur de OCR1A. En mode AUTO, cette routine de service effectue une suite de comparaisons entre **target_temp** et **temp_raw** (toutes les deux en format hexadécimal). Ces comparaisons forment un step function pour les vitesses du moteur, montré en **Annexe F**.

Les drivers (**servo_set_PWM** et **servo_set_stop**) pour ce module sont très simples. Ils s'occupent de charger la valeur 16-bit déterminée par **servo_service** (stocké en a0 et a1) au registre OCR1A.
Il est possible de calibrer le moteur afin de déterminer son vrai point neutre avec la table **servo_calibration_table**, qui peut remplacer la table **servo_speed_table** dans le mode MANUAL avec une légère modification du code source. Une calibration s'agit d'observer pour quelles valeurs d'OCR1A le moteur ne bouge pas, et d'ajuster la valeur **SERVO_STOP** en haut de **servo_module.asm**.

Branchement matériel du moteur (branché sur module M4): **PORTB, Pin 5**

5. Détails d'accès aux périphériques

5.1 Récepteur infrarouge (IR) - Module M2

Comme précisé à la section 4.1.1, le récepteur infrarouge est branché au **port E**, dont la broche numéro 7 est connectée matériellement à l'interruption **INT7.** Cette broche exécute deux fonctions: détecter un début de trame de transmission de commande et échantillonner les bits de la trame. A cet effet:

- le bit 7 du registre **DDRE** est mis à logique 0 (input)

Protocole de transmission: **RC5**

Au repos, la ligne de transmission des données (PE7) est au niveau logique 1. Le premier bit de start est communiqué par le passage de la ligne au niveau logique 0. En effet, l'interruption 7 est configurée de sorte à se déclencher à la détection d'un flanc descendant. A cet effet:

- La paire de bits **ISC71:ISC70** (Interrupt Sens Control, INT7) du registre **EICRB** (External Interrupt Command Register B) est mise à **0b10** (flanc descendant)

L'interruption 7 est activée:

- le bit 7 du registre **EIMSK** (External Interrupt Mask) est mis à 1

5.2 Capteur de température DS18B20 - Module M5

Comme précisé à la section 4.1.2, le capteur de température est branché au **port D**,au **pin 5**.

Protocole de transmission: **1-Wire**

Ce port étant bidirectionnel, il est compatible avec le protocole **1-Wire**, dans lequel la communication entre le maître et l'esclave s'effectue sur une unique ligne de données. A cet effet:

- le **bit 5** du registre **DDRD** est mis à 0 (input hi-Z)
- le **bit 5** du registre **PORTD** est mis à 0 (pull-down)

Le capteur de température possède sa propre mémoire SRAM (appelé Scratchpad) , dont le **byte 4**, à savoir le **Registre de Configuration**, stocke le code binaire associée à la résolution de mesure souhaitée.

SCRATCHPAD (POWER-UP STATE)	
Byte 0	Temperature LSB (50h) } (85°C)
Byte 1	Temperature MSB (05h) } (85°C)
Byte 2	T_H Register or User Byte 1*
Byte 3	T_L Register or User Byte 2*
Byte 4	Configuration Register*
Byte 5	Reserved (FFh)
Byte 6	Reserved
Byte 7	Reserved (10h)
Byte 8	CRC*

*Power-up state depends on value(s) stored in EE

BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
0	R1	R0	1	1	1	1	1

R1	R0	RESOLUTION (BITS)	MAX CONVERSION TIME	
0	0	9	93.75ms	($t_{CONV}/8$)
0	1	10	187.5ms	($t_{CONV}/4$)
1	0	11	375ms	($t_{CONV}/2$)
1	1	12	750ms	(t_{CONV})

Nous avons choisi une résolution de 11 bits. A cet effet, par la commande writeScratchpad, on écrit le code **0x5F** (= 0b01011111) dans le registre de configuration. Le lancement et la lecture d'une nouvelle conversion se fait grâce aux commandes convertT et readScratchpad, respectivement, à l'aide des sous-routines wire1_write et wire1_read de la librairie qui nous est fournie.

5.3 SG90 9g Micro Servo - Module M4

Le **Timer1** est configuré en mode **Fast PWM** avec **ICR1 comme valeur TOP**, ce qui permet de générer sans routine d'interruption un signal périodique sur le pin matérielle **OC1A** (**PORTB, Pin 5**).

Avec une horloge CPU de 4 MHz et un **prescaler de 8**, la fréquence du timer vaut 500kHz, soit un tick de 2 µs. Pour obtenir une période de **20ms** comme spécifiée dans la documentation du moteur SG90, il faut 10'000 ticks, d'où **ICR1 = 9999**.

La vitesse de rotation est encodée dans la durée de l'impulsion haute (Pulse-Width) pendant la période de 20ms, fixée par **OCR1A** (une valeur de 2 bytes, en ticks):

500 ticks → PW = 1,0 ms → vitesse maximale dans un sens
750 ticks → PW = 1,5 ms → presque l'arrêt (point neutre à calibrer)
1000 ticks → PW = 2,0 ms → vitesse maximale dans l'autre sens

Pour modifier la vitesse, il suffit d'écrire une nouvelle valeur dans OCR1A (la tâche du driver **servo_set_PWM**).

Pour configurer le Timer1 dans ce mode,

- le **bit 5** du **DDRB** est mis à **1** (pour faire apparaître le signal carré)
- le **TCCR1(A/B)** configure le mode non-inverseur sur OC1A, en Fast PWM avec ICR1 comme TOP (WGM1(3:0) = 0b1110), et le prescaler de 8
    - le **TCCR1A** est donc
        - WGM11 = 1, WGM10 = 0, COM1A(1:0) = 0b10 (mode non-inverseur)
    - le **TCCR1B** est donc
        - WGM13 = 1, WGM12 = 1, CS1(2:0) = 0b010 (prescaler)
- le **ICR1H:ICR1L = high(9999):low(9999)** (pour fixer la période à 20ms)
- le **OCR1AH:OCR1AL = high(SERVO_STOP):low(SERVO_STOP)** (la valeur initiale de OCR1A, avec SERVO_STOP étant la valeur calibré du point neutre en ticks)

Avec 10 vitesses manuelles (0-9) et une vitesse maximale encodé par le OCR1A à 1000 ticks, nous définissons alors la table **servo_speed_table** ci-dessous:

OCR1A (en ticks)	Mapping
SERVO_STOP (~750)	speed 0
780	speed 1
808	speed 2

835	speed 3
862	speed 4
890	speed 5
918	speed 6
945	speed 7
972	speed 8
1000	speed 9

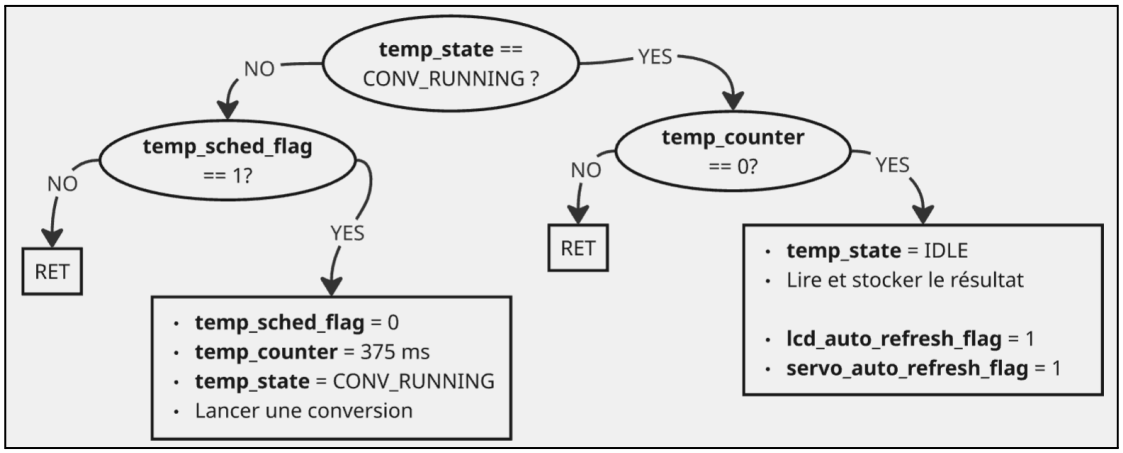
Avec 5 vitesses automatiques contrôlées par la différence entre **target_temp** et **temp_raw**, nous définissons les ticks correspondant aux vitesses du “step function” (illustré en **Annexe F**) ci-dessous:

OCR1A	Mapping	Condition (0x10 = 1ºC)
800	auto_slow	temp_raw < target_temp
850	auto_med_low	target_temp < temp_raw < target_temp + 0x10
900	auto_med	target_temp + 0x10 < temp_raw < target_temp + 0x20
950	auto_med_high	target_temp + 0x20 < temp_raw < target_temp + 0x30
1000	auto_fast	target_temp + 0x30 < temp_raw

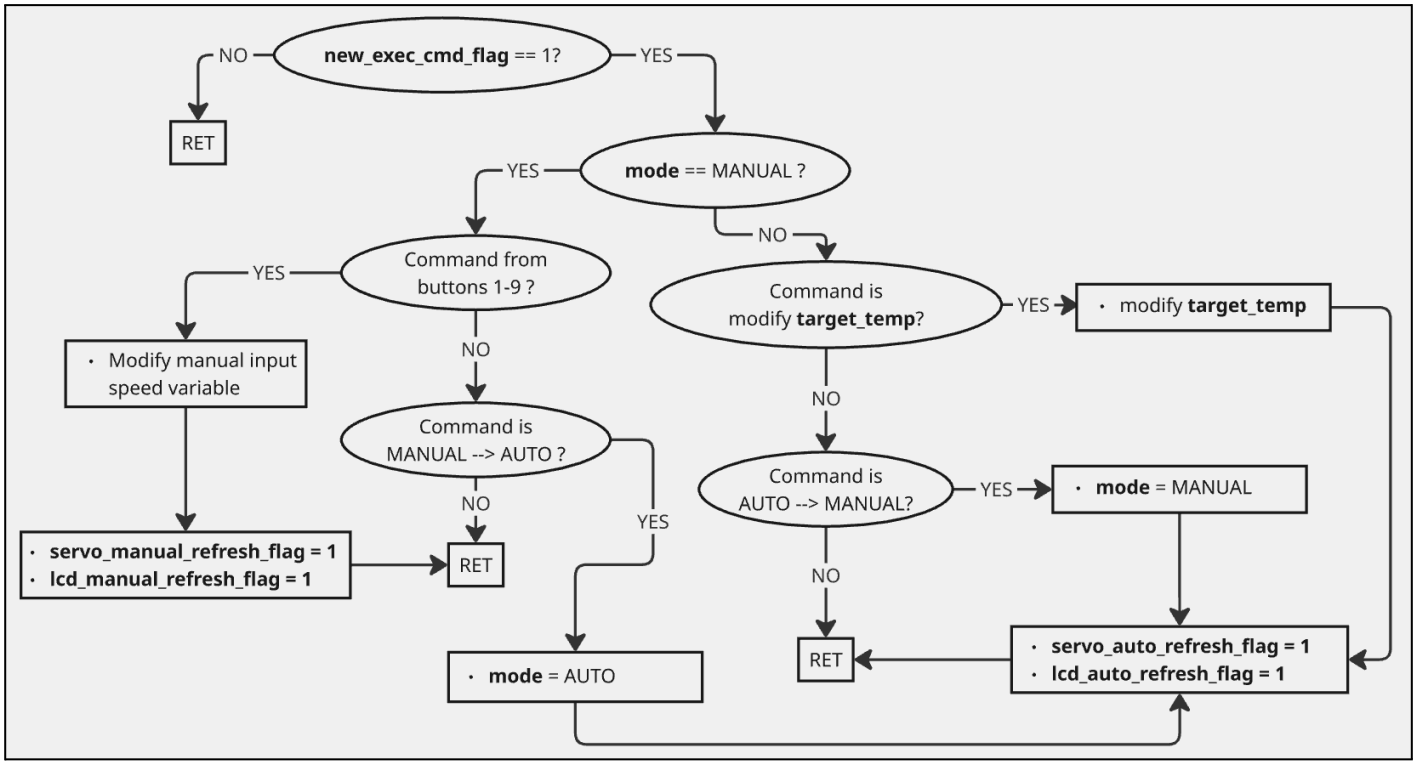
6. Annexes

6.1 Flowcharts d'exécution des services appelés dans main (excepté le scheduler)

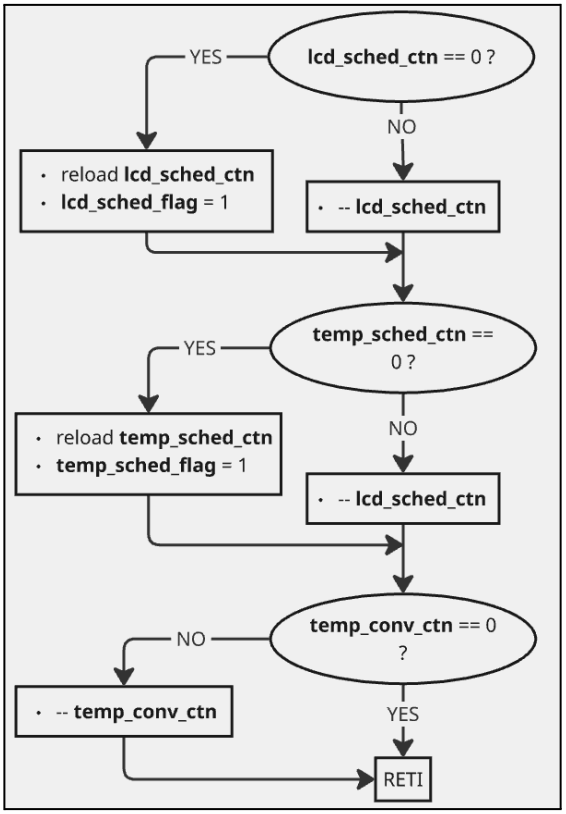
A - flowchart d'exécution du service température



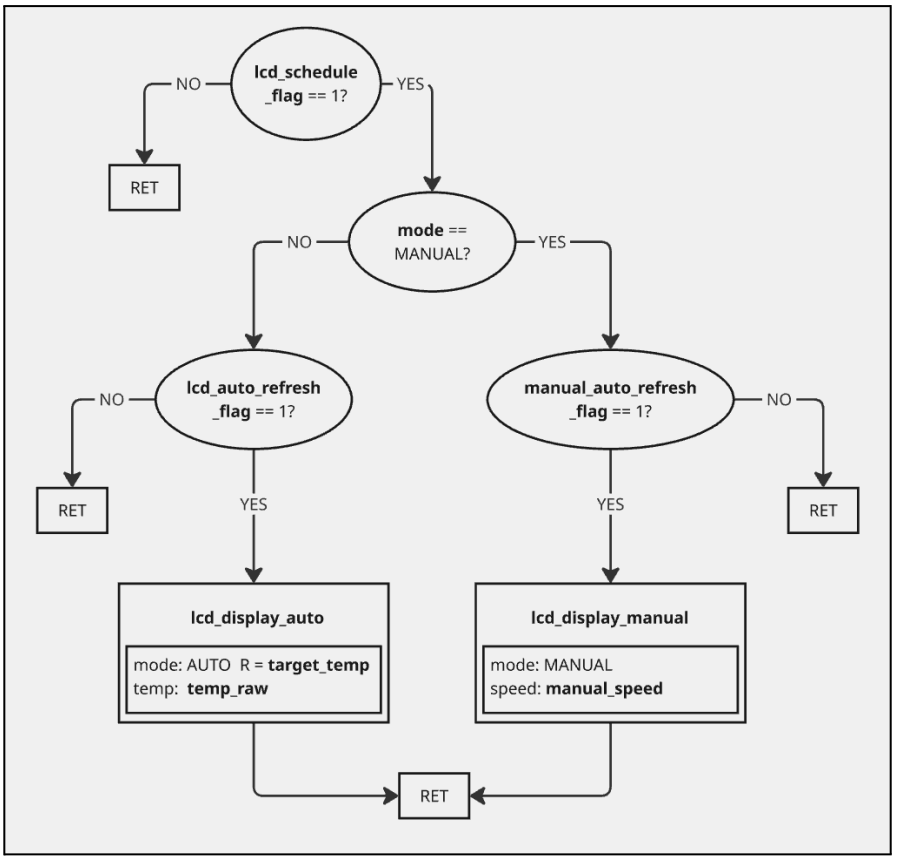
B - flowchart d'exécution du service gestion et mise à jour d'état (state_update_service)



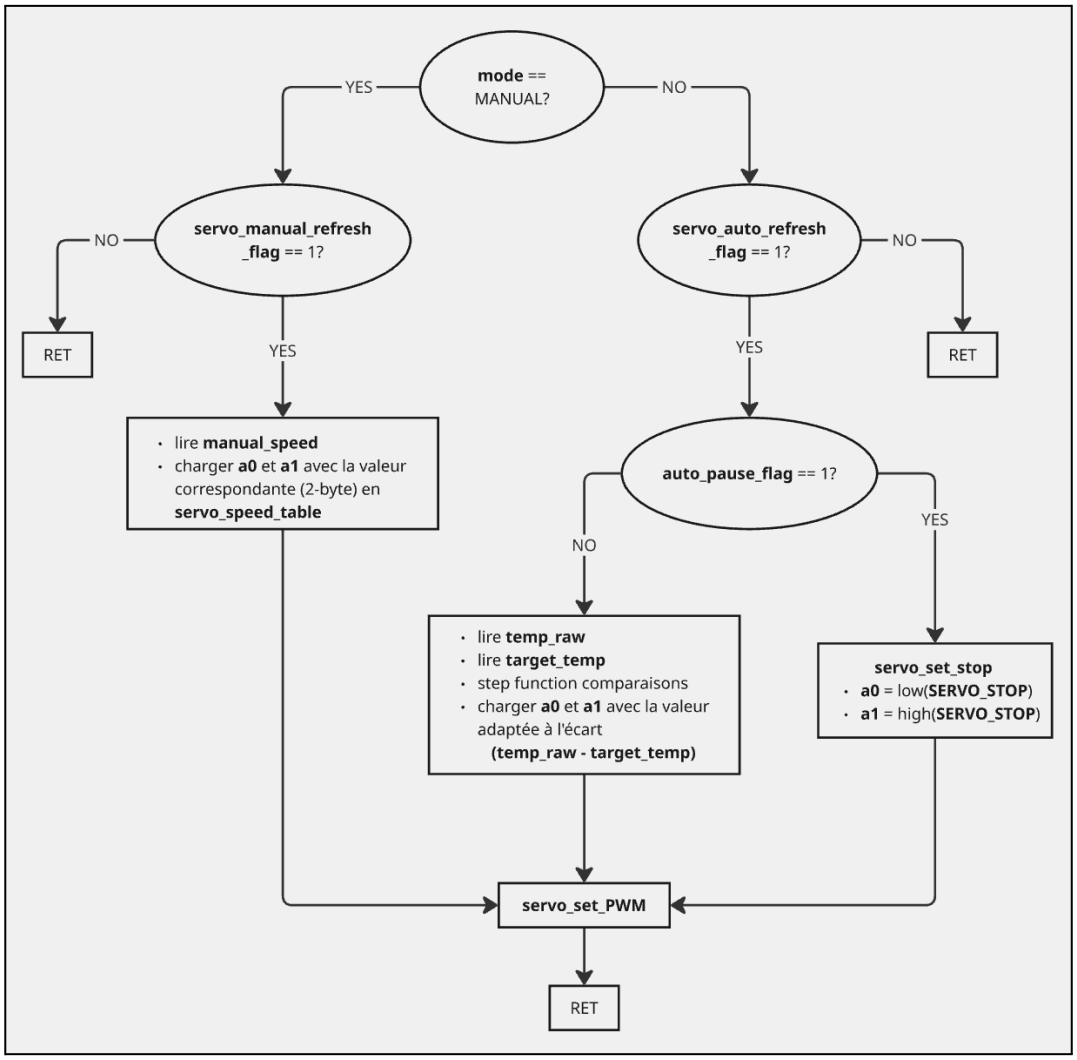
C - flowchart d'exécution du service scheduler (ISR de l'interruption COM du timer0)



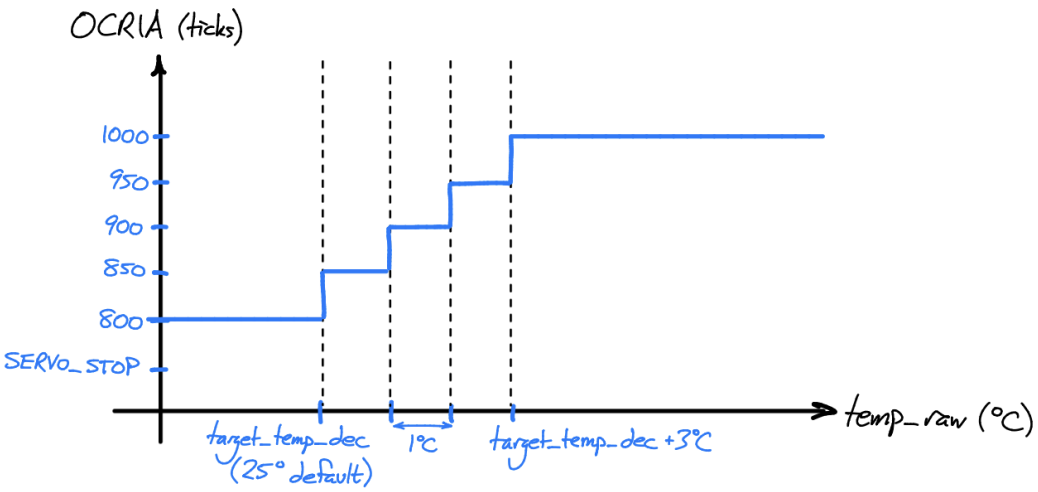
D - flowchart d'exécution du module d'affichage



E - flowchart d'exécution du module moteur



F - vitesse du moteur en fonction de la température ambiante



6.2 Code Source

Code source

project_vars.asm

```
; file: project_vars.asm
; purpose: SRAM variables
; note: this file is meant to be included afer:
;       .dseg
;       .org 0x0260
; authors: Lucas De Boi, Daniel Lelescu

; ----------------------------------------
; SCHEDULER
; ----------------------------------------
; Counters
lcd_sched_cnt:              .byte 1
temp_sched_cnt:             .byte 1
temp_conv_cnt:              .byte 1
; Flags
lcd_sched_flag:             .byte 1
temp_sched_flag:            .byte 1


; ----------------------------------------
; IR
; ----------------------------------------
; Data
command:               .byte 1
command_raw_low:       .byte 1
command_raw_hi:        .byte 1
; Flags
new_exec_cmd_flag:      .byte 1
new_raw_cmd_flag:       .byte 1
last_toggle_bit:        .byte 1

; ----------------------------------------
; TEMPERATURE
; ----------------------------------------
; Data
temp_state:                 .byte 1
temp_raw_lo:                .byte 1
temp_raw_hi:                .byte 1
target_temp:                .byte 1
target_temp_decimal:        .byte 1

; ----------------------------------------
; STATE MANAGER
; ----------------------------------------
mode:                           .byte 1
manual_speed:                   .byte 1
auto_pause_flag:                .byte 1

lcd_auto_refresh_flag:          .byte 1
lcd_manual_refresh_flag:        .byte 1

servo_auto_refresh_flag:        .byte 1
servo_manual_refresh_flag:      .byte 1

auto_servo_speed:               .byte 1
```

main.asm

```
; file: main.asm
; authors: Lucas De Boi, Daniel Lelescu
.include "m128def.inc"
.include "macros.asm"
.include "definitions.asm"


.dseg
.org 0x0260 ; zone in SRAM where variables can be directly accessed by printf and displayed
.include "project_vars.asm"

.cseg
; ========================
; table des vecteurs
; ========================
.org 0
    jmp reset

.org INT7addr ; IR command detection
    jmp ext_int7

.org OC0addr ; general timer: update of software counters and setting of sched_flags
    jmp timer0_comp_isr


.include "lcd.asm"
.include "printf.asm"
.include "wire1.asm"

; ========================
; reset
; ========================
reset:
    LDSP RAMEND
    rcall scheduler_init
    rcall ir_init
    rcall temp_init
    rcall state_init
    rcall buzzer_init
    rcall servo_init

    rcall LCD_init
    rcall LCD_clear
    rcall LCD_home
    rcall lcd_display_manual

    sei

main:
    rcall ir_service
    rcall temp_service
    rcall state_update_service
    rcall servo_service
    rcall lcd_service

    rjmp main


; ========================
; modules
; ========================
.include "scheduler_module.asm"
.include "ir_module.asm"
.include "temp_module.asm"
.include "state_manager.asm"
.include "buzzer_module.asm"
.include "servo_module.asm"
.include "lcd_module.asm"
```

ir_module.asm

```
; file: ir_module.asm
; authors: Lucas De Boi, Daniel Lelescu
; purpose: IR decoding with full reception inside INT7 ISR

; ----------------------------------------
; Local constants
; ----------------------------------------
; Same timing as reference ir_rc5.asm
; ----------------------------------------

.equ T1 = 1870


; ----------------------------------------
; ir_init
; purpose:
;   - configure PE7 as input
;   - configure INT7 on falling edge
;   - initialize IR variables
; ----------------------------------------
ir_init:
    ; PE7 input, no pull-up
    cbi     DDRE, IR ; IR is defined as pin 7 in definitions.asm
    cbi     PORTE, IR

    ; INT7 on falling edge to detect RC5 transmission start bit
    ; ISC71:ISC70 = 0b10
    ldi     w, (1<<ISC71)
    out     EICRB, w

    ; initialize IR variables
    ldi     w, 0
    sts     command, w
    sts     command_raw_low, w
    sts     command_raw_hi, w
    sts     new_exec_cmd_flag, w
    sts     new_raw_cmd_flag, w
    sts     last_toggle_bit, w

    ; clear pending INT7 flag
    ldi     w, (1<<INTF7)
    out     EIFR, w

    ; enable INT7
    ldi     w, (1<<INT7)
    out     EIMSK, w

    ret


; ----------------------------------------
; ext_int7
; purpose:
;   - receive the full 14-bit RC5 frame inside the ISR
;   - publish raw reception complete
; ----------------------------------------
ext_int7:
    ; save context
    push    w
    in      w, SREG
    push    w
    push    b0
    push    b1
    push    b2

    ; disable INT7 during decoding
    in      w, EIMSK
    andi    w, 0b01111111
    out     EIMSK, w

    ; initialize reception buffer
    ldi     w, 0
    sts     command_raw_low, w
    sts     command_raw_hi, w
```

```
    sts     new_raw_cmd_flag, w

    CLR2    b1, b0
    ldi     b2, 14

    ; same timing as reference file
    WAIT_US (T1/4)

ir_decode_loop:
    P2C     PINE, IR
    ROL2    b1, b0
    WAIT_US (T1-4)
    DJNZ    b2, ir_decode_loop

    ; store raw 14-bit reception
    sts     command_raw_low, b0
    sts     command_raw_hi, b1

    ; publish raw reception complete
    ldi     w, 1
    sts     new_raw_cmd_flag, w

    ; restore context
    pop     b2
    pop     b1
    pop     b0
    pop     w
    out     SREG, w
    pop     w
    reti


; ----------------------------------------
; ir_service
; purpose:
;   - if a raw 14-bit reception is complete, do the post-processing
;   - publish final command
;   - set new_exec_cmd_flag = 1
;   - re-enable INT7
; ----------------------------------------
ir_service:
    lds     w, new_raw_cmd_flag
    tst     w
    breq    ir_service_done

    ; consume raw-ready flag
    ldi     w, 0
    sts     new_raw_cmd_flag, w

    ; anti-corruption/debouncing measure, before rising/falling edge detection
    lds     a0, command_raw_hi      ; (00)(2x start bit)(toggle bit)(2x device address bits)
    mov     a1, a0
    andi    a1, 0b00100000          ; isolating start bit 1
    brne    ir_service_done

    ; rising/falling edge detection for toggle bit
    andi    a0, (1<<3)              ; isolating new toggle bit
    lds     a1, last_toggle_bit
    cp      a0, a1                  ; comparing toggle bits
    breq    ir_service_done         ; same toggle = held down button, skip to done

    sts     last_toggle_bit, a0 ; if unique cmd, update last toggle bit

    ; same post-processing as reference ir_rc5.asm
    lds     a0, command_raw_low
    com     a0
    sts     command, a0

    ; publish final command
    ldi     w, 1
    sts     new_exec_cmd_flag, w

ir_service_done:
    ; clear pending INT7 flag and re-enable INT7
    ldi     w, (1<<INTF7)
    out     EIFR, w
```

```

    in      w, EIMSK
    ori     w, (1<<INT7)
    out     EIMSK, w

    ret
```

temp_module.asm

```
; file: temp module
; authors: Lucas De Boi, Daniel Lelescu

; file: temp_module.asm
; purpose: all constants and subroutines related to the temperature module

; ----------------------------------------
; Local constants
; ----------------------------------------

; 1-Wire ROM / function commands are defined in wire1.asm


; DS18B20 configuration byte
; 11-bit resolution -> config = 0x5F
.equ TEMP_RES = 0x5F

; Internal states
.equ IDLE           = 0
.equ CONV_RUNNING   = 1

; Scheduler-based conversion wait
; One scheduler tick ~= 9.984 ms
; 41 ticks ~= 409.3 ms (a bit of margin from the conversion time for 11 bit resolution which is 375 ms)
.equ TEMP_CONV_TICKS = 41

; Overheat threshold
; .equ TEMP_OVERHEAT_RAW   = 0x01E0


; ----------------------------------------
; temp_init
; purpose:
;   - initialize 1-Wire interface
;   - configure the temp sensor to 11-bit resolution (0.25C)
;   - initialize temperature variables
; ----------------------------------------
temp_init:
    rcall   wire1_init

    ; initialize variables
    ldi     w, 0
    sts     temp_state, w
    sts     temp_raw_lo, w
    sts     temp_raw_hi, w

    ldi     w, 0x90
    sts     target_temp, w ; 25C default target
    ldi     w, 25
    sts     target_temp_decimal, w

    ; Configure DS18B20 resolution to 11 bits
    ; Write Scratchpad: TH, TL, Config
    rcall wire1_reset
    CA wire1_write, skipROM
    CA wire1_write, writeScratchpad
    CA wire1_write, 0x00          ; TH dummy value
    CA wire1_write, 0x00          ; TL dummy value
    CA wire1_write, TEMP_RES

    ret


; ----------------------------------------
```

```
; temp_service
; purpose:
;   - if IDLE and temp_sched_flag = 1:
;       start a conversion and load temp_conv_cnt
;   - if CONV_RUNNING and temp_conv_cnt = 0:
;       read temperature (and update overheat_flag)
; ----------------------------------------
temp_service:
    lds w, temp_state
    tst w
    breq temp_idle_part

    ; --------------------------------
    ; conversion running
    ; --------------------------------
    lds w, temp_conv_cnt
    tst w
    brne temp_service_done

    ; conversion finished -> read scratchpad

    rcall wire1_reset
    CA wire1_write, skipROM
    CA wire1_write, readScratchpad

    rcall wire1_read
    sts temp_raw_lo, a0

    rcall wire1_read
    sts temp_raw_hi, a0

    ; back to IDLE
    ldi w, IDLE
    sts temp_state, w

    ; update lcd and servo AUTO refresh flags
    ldi w, 1
    sts lcd_auto_refresh_flag, w
    sts servo_auto_refresh_flag, w

    rjmp temp_service_done

    ; --------------------------------
    ; idle state
    ; --------------------------------
temp_idle_part:
    lds w, temp_sched_flag
    tst w
    breq temp_service_done

    ; consume scheduling flag
    ldi w, 0
    sts temp_sched_flag, w

    ; start a new conversion
    rcall wire1_reset
    CA wire1_write, skipROM
    CA wire1_write, convertT

    ; load relative wait counter
    ldi w, TEMP_CONV_TICKS
    sts temp_conv_cnt, w

    ; enter running state
    ldi w, CONV_RUNNING
    sts temp_state, w

temp_service_done:
    ret
```

scheduler_module.asm

```
; file: scheduler.asm
; authors: Lucas De Boi, Daniel Lelescu

; file: scheduler_module.asm
; purpose: periodic software scheduler using Timer0 compare match

; ----------------------------------------
; Local constants
; ----------------------------------------
; Timer0:
;   f_clk = 4 MHz
;   prescaler = 1024
;   tick = 256 us
;
; CTC ("Clear Timer on Compare Match) with OCR0 (Ouput Compare Register) = 38:
;   period = (38+1)*256 us = 9984 us ~= 10 ms
; ----------------------------------------

.equ SCHED_OCR0              = 38

; software periods expressed in scheduler ticks
.equ LCD_SCHED_RELOAD        = 10      ; ~100 ms
.equ TEMP_SCHED_RELOAD     = 100     ; ~ 1 ms

; ----------------------------------------
; scheduler_init
; purpose:
;   - configure Timer0 in CTC (Clear Timer on Compare Match) mode
;   - enable compare match interrupt
;   - initialize software counters and flags
; ----------------------------------------
scheduler_init:
    ; initialize software counters
    ldi     w, LCD_SCHED_RELOAD
    sts     lcd_sched_cnt, w

    ldi     w, TEMP_SCHED_RELOAD
    sts     temp_sched_cnt, w

    ldi     w, 0
    sts     temp_conv_cnt, w
    sts     lcd_sched_flag, w
    sts     temp_sched_flag, w

    ; Timer0 CTC mode, OCR0 = 38
    ldi     w, SCHED_OCR0
    out     OCR0, w

    ; WGM01 = 1 -> CTC
    ; CS02:CS00 = 111 -> prescaler 1024
    ldi     w, (1<<WGM01) + (1<<CS02) + (1<<CS01) + (1<<CS00)
    out     TCCR0, w

    ; clear any pending compare flag
    ldi     w, (1<<OCF0)
    out     TIFR, w

    ; enable Timer0 compare interrupt
    in      w, TIMSK
    ori     w, (1<<OCIE0)
    out     TIMSK, w

    ret


; ----------------------------------------
; timer0_comp_isr
; purpose:
;   periodic scheduler tick (~10 ms)
; ----------------------------------------
timer0_comp_isr:
    push    w
    in      w, SREG
    push    w
```

```

    ; --------------------------------
    ; LCD scheduler counter
    ; --------------------------------
    lds     w, lcd_sched_cnt
    tst     w
    breq    lcd_cnt_reload

    dec     w
    sts     lcd_sched_cnt, w
    brne    temp_sched_part

lcd_cnt_reload:
    ldi     w, LCD_SCHED_RELOAD
    sts     lcd_sched_cnt, w

    ldi     w, 1
    sts     lcd_sched_flag, w

temp_sched_part:
    ; --------------------------------
    ; Temperature scheduler counter
    ; --------------------------------
    lds     w, temp_sched_cnt
    tst     w
    breq    temp_cnt_reload

    dec     w
    sts     temp_sched_cnt, w
    brne    temp_conv_part

temp_cnt_reload:
    ldi     w, TEMP_SCHED_RELOAD
    sts     temp_sched_cnt, w

    ldi     w, 1
    sts     temp_sched_flag, w

temp_conv_part:
    ; --------------------------------
    ; Temperature conversion countdown
    ; --------------------------------
    lds     w, temp_conv_cnt
    tst     w
    breq    scheduler_isr_done

    dec     w
    sts     temp_conv_cnt, w

scheduler_isr_done:
    pop     w
    out     SREG, w
    pop     w
    reti
```

state_manager.asm

```
; file: state_manager.asm
; authors: Lucas De Boi, Daniel Lelescu

; ----------------------------------------
; Local constants
; ----------------------------------------
; Mode symbols
.equ MODE_MANUAL = 0
.equ MODE_AUTO = 1

; Command constants (determined empirically)
.equ CMD_MODE_MANUAL = 0x20
.equ CMD_MODE_AUTO = 0x21
.equ CMD_PAUSE_PLAY = 0x0d
.equ CMD_PLUS = 0x10
.equ CMD_MINUS = 0x11

state_init:
    ldi     w, MODE_MANUAL
    sts     mode, w

    ldi     w, 0
    sts     manual_speed, w
    sts     auto_pause_flag, w

    sts     lcd_auto_refresh_flag, w
    sts     lcd_manual_refresh_flag, w
    sts     servo_auto_refresh_flag, w
    sts     servo_manual_refresh_flag, w

    ret


state_update_service:
    ; Check if there's a new command to process
    lds     w, new_exec_cmd_flag
    tst     w
    brne    PC+2
    rjmp    state_update_done

    ; If there is a new command to process, we consume the new_exec_cmd_flag and process the command
    ldi     w, 0
    sts     new_exec_cmd_flag, w

    rcall buzzer_beep


    ; we first check what mode we're in and according to that modify the state variables according to the command
    lds     w, command

    lds     r17, mode
    cpi     r17, MODE_MANUAL
    breq    state_manual_part

state_auto_part:
    JK      w, CMD_PLUS, temp_plus
    JK      w, CMD_MINUS, temp_minus
    cpi     w, CMD_PAUSE_PLAY  ; Is it the pause/play command?
    brne    cmd_check_auto_to_manual ; If no, we go on to check if it's the manual mode button

    ; if pause/play, we complement the auto_pause_flag, and return
    lds     r17, auto_pause_flag
    ldi     r18, 1
    eor     r17, r18
    sts     auto_pause_flag, r17

    rjmp state_auto_update_done

    ; adding 1C to target temp
    temp_plus:

    lds     w, target_temp
    ADDI    w, 0x10
    sts     target_temp, w ; updating target_temp in hex, for servo
```

```

    lds     w, target_temp_decimal
    ADDI    w, 1
    sts     target_temp_decimal, w ; updating target_temp in dec, for display

    STI     command, 1 ; consuming the plus
    rjmp    state_auto_update_done

    ; substracting 1C from target tepm
    temp_minus:

    lds     w, target_temp
    SUBI    w, 0x10
    sts     target_temp, w ; updating target_temp in hex, for servo

    lds     w, target_temp_decimal
    SUBI    w, 1
    sts     target_temp_decimal, w ; updating target_temp in dec, for display

    STI     command, 1 ; consuming the minus
    ; fall through to auto_update_done

    state_auto_update_done:
    ldi     r17, 1
    sts     lcd_auto_refresh_flag, r17
    sts     servo_auto_refresh_flag, r17

    rjmp    state_update_done

cmd_check_auto_to_manual:
    cpi     w, CMD_MODE_MANUAL ; Is it the manual mode button?
    brne    state_update_done ; If no, then it's an irrelevant button has been pressed and we do nothing

    ; If yes, we set the mode to manual
    ldi     r17, MODE_MANUAL
    sts     mode, r17

    ldi     r17, 1
    sts     lcd_manual_refresh_flag, r17
    sts     servo_manual_refresh_flag, r17

    rjmp    state_update_done

state_update_done: ; closer label for breq
    ret

state_manual_part:
    ; if its equal or greater than 10, then it's not a speed modification command, but of the other two: (pause/play) OR (manual -> auto)
    cpi     w, 10
    brsh    cmd_check_pause_play

    sts     manual_speed, w

    ldi     r17, 1
    sts     lcd_manual_refresh_flag, r17
    sts     servo_manual_refresh_flag, r17
    rjmp    state_update_done

cmd_check_pause_play:
    cpi     w, CMD_PAUSE_PLAY ; Is it the pause/play button?
    brne    cmd_check_manual_to_auto

    ldi     r17, 0
    sts     manual_speed, r17

    ldi     r17, 1
    sts     lcd_manual_refresh_flag, r17
    sts     servo_manual_refresh_flag, r17
    rjmp    state_update_done

cmd_check_manual_to_auto:
    cpi     w, CMD_MODE_AUTO ; Is it the manual --> auto button?
    brne    state_update_done ; If no, then irrelevant command

    ; If yes, we change the mode
```

```
    ldi     r17, MODE_AUTO
    sts     mode, r17

    ; If we were in AUTO and paused it, then changed to MANUAL, we would want the pause to not still be active when we go back
    ldi     r17, 0
    sts     auto_pause_flag, r17

    ldi     r17, 1
    sts     lcd_auto_refresh_flag, r17
    sts     servo_auto_refresh_flag, r17
    rjmp state_update_done
```

servo_module.asm

```
; file: servo_module.asm
; authors: Lucas de Boi, Daniel Lelescu
; purpose: control of servo motor via PWM lookup table, to emulate fan

; Architecture:
;   servo_init    : Bloc 0 (driver) - Timer1 hardware PWM initialisation
;   servo_set_ocr : Bloc 0 (driver) - updates OCR1A pulse width
;   servo_manager : Bloc 1 (manager) - checks refresh flags, calls driver
;
; Hardware PWM  Timer1, Fast PWM, ICR1 as TOP:
;   - Output pin: OC1A = PB5 (hardware-fixed, set as output in DDRB)
;   - Prescaler 8 @ 4MHz -> tick = 2 us
;   - ICR1 = 9999 -> period = 10000 * 2 us = 20 ms
;   - OCR1A sets pulse width: OCR1A ticks * 2 us = pulse in us
;
; Modified SG90 pulse mapping (tick = 2 us):
;   500 ticks = 1000 us = 1.0 ms -> full speed one direction
;   750 ticks = 1500 us = 1.5 ms -> stopped (neutral)
;  1000 ticks = 2000 us = 2.0 ms -> full speed other direction
;
; Speed 1-9 lookup table maps to OCR1A tick values (500-1000).
;
;

; ----------------------------------------
; Local constants
; ----------------------------------------
.equ SERVO_ICR1     = 9999  ; TOP -> 20 ms period
.equ SERVO_STOP     = 769  ; calibrated neutral/stop
.equ SERVO_MIN      = 500   ; 1.0 ms -> full speed one direction
.equ SERVO_MAX      = 1000  ; 2.0 ms -> full speed other direction


; ----------------------------------------
; servo_init
; Configure Timer1 for Fast PWM on OC1A (PB5).
; Call once during initialisation, before sei.
; ----------------------------------------
servo_init:

    ; OC1A (PB5) must be set as output for hardware PWM to appear on pin
    sbi     DDRB, 5

    ; reset timer counter
    ldi     w, 0
    out     TCNT1H, w
    out     TCNT1L, w
    sts     auto_servo_speed, w ; initializing testing variable

    ; --- ICR1 = 9999 (sets 20ms period) ---
    ldi     w, high(SERVO_ICR1)
    out     ICR1H, w
    ldi     w, low(SERVO_ICR1)
    out     ICR1L, w

    ; --- OCR1A = SERVO_STOP (motor starts stopped) ---
    ldi     w, high(SERVO_STOP)
    out     OCR1AH, w
```

```
    ldi     w, low(SERVO_STOP)
    out     OCR1AL, w

    ; --- TCCR1A ---
    ; COM1A1:COM1A0 = 1:0 -> non-inverting Fast PWM on OC1A
    ;   (OC1A set at BOTTOM, cleared on compare match)
    ; WGM11:WGM10   = 1:0 -> Fast PWM mode 14 (ICR1 as TOP), combined with TCCR1B
    ldi     w, (1<<COM1A1) | (1<<WGM11)
    out     TCCR1A, w

    ; --- TCCR1B ---
    ; WGM13:WGM12 = 1:1 -> Fast PWM mode 14 (ICR1 as TOP)
    ; CS12:CS11:CS10 = 0:1:0 -> prescaler 8
    ldi     w, (1<<WGM13) | (1<<WGM12) | (1<<CS11)
    out     TCCR1B, w

    ret


servo_service:
    push    a0
    push    a1
    push    w
    push    b0

    ; checking mode
    lds     w, mode
    cpi     w, MODE_MANUAL
    brne    PC+2
    rjmp    servo_manual

    ; --- AUTO ---
    servo_auto:
    lds     w, servo_auto_refresh_flag
    tst     w
    brne    PC+2
    rjmp    servo_service_done

    ; clearing auto_refresh_flag in case of 1
    clr     w
    sts     servo_auto_refresh_flag, w

    ; checking pause_flag
    lds     w, auto_pause_flag
    tst     w
    breq    PC+2
    rjmp    servo_set_stop


    ; map integer degrees (temp_raw) to OCR tick value
    lds     a1, temp_raw_hi ; not needed to be analyzed until temperatures > 32C, as per DS18B20 decoding
    lds     a0, temp_raw_lo

    lds     b0, target_temp

    cp      a0, b0 ; (temp < target temp) - 1
    brlo    servo_auto_slow
    ADDI    b0, 0x10
    cp      a0, b0 ; (target + 1C) - 2
    brlo    servo_auto_med_low
    ADDI    b0, 0x10
    cp      a0, b0 ; (target + 2C) - 3
    brlo    servo_auto_med
    ADDI    b0, 0x10
    cp      a0, b0 ; (target + 3C) - 4
    brlo    servo_auto_med_high

    servo_auto_fast: ; (target + 4C) - 5
        ldi     a1, high(1000)
        ldi     a0, low(1000)
        STI     auto_servo_speed, 5
        rjmp    servo_set_PWM

    servo_auto_med_high:
        ldi     a1, high(950)
        ldi     a0, low(950)
```

```
        STI     auto_servo_speed, 4
        rjmp    servo_set_PWM

    servo_auto_med:
        ldi     a1, high(900)
        ldi     a0, low(900)
        STI     auto_servo_speed, 3
        rjmp    servo_set_PWM

    servo_auto_med_low:
        ldi     a1, high(850)
        ldi     a0, low(850)
        STI     auto_servo_speed, 2
        rjmp    servo_set_PWM

    servo_auto_slow:
        ldi     a1, high(800)
        ldi     a0, low(800)
        STI     auto_servo_speed, 1
        rjmp    servo_set_PWM

    ; --- MANUAL ---
    servo_manual:
    lds     w, servo_manual_refresh_flag
    tst     w
    breq    servo_service_done

    ; clearing manual_refresh_flag in case of 1
    clr     w
    sts     servo_manual_refresh_flag, w

    ; table lookup: manual_speed 1-9 -> OCR tick value
    lds     w, manual_speed

    ldi     zl, low(2*servo_speed_table)
    ldi     zh, high(2*servo_speed_table)
    ; ! decomment and comment above 2 lines to set calibration table !
    ;ldi        zl, low(2*servo_calibration_table)
    ;ldi        zh, high(2*servo_calibration_table)

    lsl     w                       ; byte offset = index * 2
    add     zl, w
    brcc    PC+2
    inc     zh

    lpm     a0, z+                  ; low byte first (.db layout)
    lpm     a1, z                   ; high byte

    rjmp    servo_set_PWM

; --- DRIVER ---
servo_set_stop:
    ldi     a1, high(SERVO_STOP)
    ldi     a0, low(SERVO_STOP)


; changes PWM via OCR1A, input 16bit tick value e.g high(750):low(750)
servo_set_PWM:
    out     OCR1AH, a1
    out     OCR1AL, a0
    ; drop down into service_done

servo_service_done:
    pop     b0
    pop     w
    pop     a1
    pop     a0
    ret


; ----------------------------------------
; servo_speed_table
; Manual speed 1-9 -> OCR1A tick values (2 us/tick)
; speed 1 =  500 ticks = 1.0 ms  (fast one direction)
; speed 5 =  750 ticks = 1.5 ms  (stopped)
; speed 9 = 1000 ticks = 2.0 ms  (fast other direction)
```

```
; Stored low byte first to match lpm a0,z+ / lpm a1,z
; ----------------------------------------
servo_speed_table:
    .db low(SERVO_STOP), high(SERVO_STOP) ; speed 0
    .db low(780),  high(780)    ; speed 1 -- slowest
    .db low(808),  high(808)    ; speed 2
    .db low(835),  high(835)    ; speed 3
    .db low(862),  high(862)    ; speed 4
    .db low(890),  high(890)    ; speed 5
    .db low(918),  high(918)    ; speed 6
    .db low(945),  high(945)    ; speed 7
    .db low(972),  high(972)    ; speed 8
    .db low(1000), high(1000)   ; speed 9 -- fastest

servo_calibration_table: ; to find SERVO_STOP
    .db low(765),  high(765)    ; -- lower bound, 0
    .db low(766),  high(766)    ; 1
    .db low(767),  high(767)    ; 2
    .db low(768),  high(768)    ; 3
    .db low(769),  high(769)    ; 4
    .db low(770),  high(770)    ; 5
    .db low(771),  high(771)    ; 6
    .db low(772),  high(772)    ; 7
    .db low(773),  high(773)    ; 8
    .db low(774),  high(774)    ; -- upper bound, 9
```

lcd_module.asm

```
; file: lcd_module.asm
; authors: Lucas de Boi, Daniel Lelescu
; purpose: visual interface of the application

lcd_service:

    ; checking lcd_sched_flag
    lds w, lcd_sched_flag
    tst w
    brne PC+2
    rjmp lcd_service_done

    ; consuming lcd_sched_flag if 1
    ldi w, 0
    sts lcd_sched_flag, w

    ; checking mode
    lds w, mode
    cpi w, MODE_MANUAL
    breq lcd_service_manual


    ; lcd_service_auto
    lds w, lcd_auto_refresh_flag
    tst w
    brne PC+2
    rjmp lcd_service_done

    ; consuming auto_refresh_flag if 1
    ldi w, 0
    sts lcd_auto_refresh_flag, w
    rcall LCD_clear
    rjmp lcd_display_auto


    lcd_service_manual:
    lds w, lcd_manual_refresh_flag
    tst w
    brne PC+2
    rjmp lcd_service_done

    ; consuming manual_refresh_flag if 1
    ldi w, 0
    sts lcd_manual_refresh_flag, w
    rcall LCD_clear
```

```
    rjmp lcd_display_manual

; --- drivers ---
lcd_display_auto:
    lds     a0, temp_raw_lo
    lds     a1, temp_raw_hi

    rcall LCD_home
    PRINTF LCD
    .db "mode: AUTO R=",FDEC,target_temp_decimal,"C",LF,0
    PRINTF LCD
    .db "temp: ",FFRAC2+FSIGN,a,4,$42,"C   ",0

    ;auto_servo_speed displayed on LEDs, abanadoned idea
    ;lds        w, auto_servo_speed

    rjmp lcd_service_done

lcd_display_manual:
    rcall LCD_home
    PRINTF LCD
    .db "mode: MANUAL",LF,0
    PRINTF LCD
    .db "Speed: ",FHEX,low(manual_speed),0

lcd_service_done:
    ret
```

buzzer_module.asm

```
; file: buzzer_module.asm
; authors: Lucas de Boi, Daniel Lelescu
; purpose: control of buzzer module, with goal of response to IR inputs

buzzer_init:
    sbi     DDRE, SPEAKER
    cbi     PORTE, SPEAKER
    ret


buzzer_beep: ; makes the buzzer beep, to be called when a button is pressed
    push    a0
    push    a1
    push    w

    ldi     a0, 20
    buzz:
        sbi     PORTE, SPEAKER
        WAIT_US 1500
        cbi     PORTE, SPEAKER
        WAIT_US 1500
        dec     a0
        brne    buzz ; loop until a0 = 0

    pop     w
    pop     a1
    pop     a0

    ret
```